SHL Assessment Recommendation System: Solution Approach

1. Problem Overview and Objective

The objective of this assignment was to build an intelligent recommendation system to help hiring managers and recruiters find the most relevant SHL assessments from the product catalog based on a natural language query or a job description. The system must retrieve the most relevant pre-existing "Individual Test Solutions," balance the recommendations across domains (e.g., technical vs. personality/behavioral skills) and return exactly 5–10 highly tailored assessments in a tabular structured format.

2. System Architecture & Pipeline

The solution leverages a **Retrieval-Augmented Generation (RAG) architecture** with a two-stage retrieve-and-rerank pipeline.

Step 2.1: Data Ingestion and Processing

- **Web Scraping:** A custom, robust web scraper (`src/crawler/shl_scraper.py`) was built using `BeautifulSoup` to traverse the SHL site map and extract product listings.
- **Data Structuring:** The scraper extracted over 377 "Individual Test Solutions", ignoring pre-packaged solutions. For each test, we stored the Name, URL, Description, Duration, Test Type (e.g., Knowledge & Skills vs. Personality & Behavior), and Remote/Adaptive capabilities. The structured document was cleaned and serialized into `data/processed/assessments_clean.json`.

Step 2.2: Embedding and Vector Storage (Retrieval)

- **Embedding Model:** We utilized the lightweight and highly performant `sentence-transformers/all-MiniLM-L6-v2` model. This model converts the combined textual representations composed of each assessment's Name, Type, Duration, and Description into high-dimensional semantic vectors.
- **Vector Database (FAISS):** The resulting embeddings were indexed using Facebook AI Similarity Search (FAISS) using the L2 distance metric. This ensures that the first-pass retrieval operates at extremely low latency (sub-millisecond search time) while accurately capturing the semantic intent of the user's query compared to traditional keyword searches.

Step 2.3: Second-Stage Reranking (Generative AI)

- **Candidate Pool:** Upon receiving a query, the FAISS engine retrieves an over-sampled candidate pool of the **Top 20** most semantically similar assessments.
- **LLM Reasoning (Google Gemini Gemini API):** The candidate pool is passed to the Gemini generative model. The model acts as a highly intelligent "reranker." The LLM is provided with a strict system prompt instructing it to:

    1. Filter out candidate assessments that violate strict constraints (e.g., "must be less

than 30 minutes").
2. Enforce recommendation balance. Specifically, if a job description implies both technical competencies (e.g., Python/SQL) and behavioral traits (e.g., communication/teamwork), the LLM intelligently picks a diverse subset representing *Test Type K* (Knowledge) and *Test Type P* (Personality/Behavior).
3. Output the final Top 5-10 most relevant assessments.

Step 2.4: Application Layer (Backend & Frontend)

- **FastAPI Backend:** A robust, fully-typed REST API built with FastAPI validates incoming JSON requests natively and manages the pipeline asynchronously.
- **Streamlit Frontend:** A sleek, interactive web interface allows non-technical users to paste job descriptions and receive actionable, clickable recommendation cards.

---

3. Evaluation & Performance Optimization

Evaluation was a critical driver behind our design choices. The system was validated entirely using the automated `generate_predictions.py` offline pipeline over the provided test and train query sets.

3.1 Mean Recall@10 Optimization Strategy

Our priority was maximizing Mean Recall@10 as per the scoring rubric. Initially, testing a simple single-stage semantic search (FAISS directly to top-10) yielded suboptimal recall for complex, multi-domain queries (e.g., needing both a Python test and a Cognitive test). Pure semantic similarity often clustered around just one conceptual domain of the prompt.

Optimization 1: Over-fetching (Top-K Expansion) By artificially inflating the first-stage retrieval size (`k=20`), we ensured that both primary elements (e.g., Data Science) and secondary elements (e.g., Leadership/Communication) of the user's query surfaced in the candidate pool.

Optimization 2: LLM as a Reranker vs. Generator Instead of having the LLM creatively generate content, we scoped its role purely to reranking. We dynamically inject the 20 pre-fetched candidate JSON objects directly into the prompt and demand structured JSON output. This completely eliminated "vibe-coding" or hallucinated URLs. The LLM acts entirely as a zero-shot logical classifier.

3.2 Handling Domain Balancing Requirement

Queries spanning multiple domains were uniquely solved via the reranking prompt engineering. The LLM was explicitly instructed how to identify "Knowledge & Skills" versus "Personality & Behavior" assessments from the candidate schema. This contextual injection ensured our final set of 10 items was balanced proportionally to the user's job description.

4. Conclusion and Tech Stack Summary

The resulting system successfully scales with excellent performance, strong reasoning capabilities, and an optimized pipeline preventing hallucinations.

- **Core Scraping:** BeautifulSoup4, Requests

- **Vector Search:** FAISS, Sentence-Transformers (all-MiniLM-L6-v2)
- **Generative AI:** Google Gemini API (gemini-1.5-flash)
- **Serving:** FastAPI, Uvicorn, Streamlit
- **Deployment:** Render (API), Streamlit Community Cloud (Frontend)